

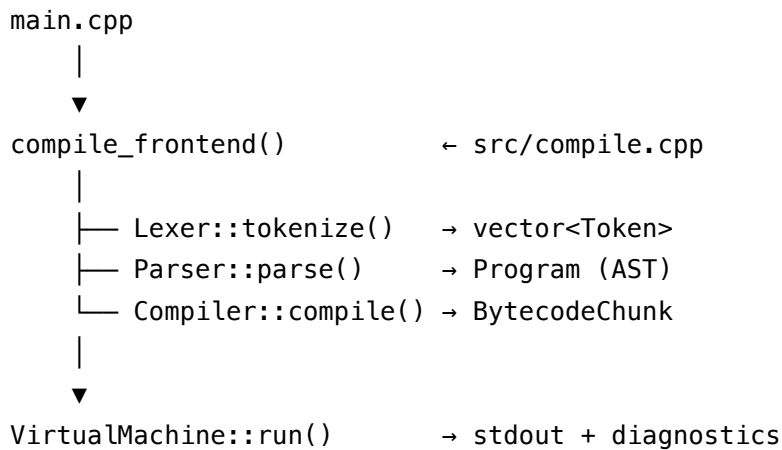
CVM++ Architecture and Workflow Guide



::: infobox{What this guide covers} **Runtime workflow only:** what each stage reads and writes, how the VM executes bytecode, opcode reference, and two full execution traces. For setup and glossary, see the **Project and Build** PDF. :::

End-to-end flow

USER runs: `./build/cvmpp script.cvm`



Stage	Files	Input	Output
Lexer	lexer.cpp	Source string	Tokens
Parser	parser.cpp	Tokens	Program AST
Compiler	compiler.cpp	AST	BytecodeChunk
VM	vm.cpp	Bytecode	Output lines

Entry points

Mode	Trigger	Notes
File	cvmpp file.cvm	Fresh globals each run
REPL	cvmpp	VmSession keeps variables
Debug	-d	Token + AST + bytecode tables
Compile-only	-c	Stops before VM

Lexer

Scans left-to-right: skips whitespace and `//` comments; emits Integer, keywords, identifiers, multi-char operators (`==`, `<=`, `...`), punctuation, Eof.

Failures (phase LEXER): bad character, integer overflow, null byte.

Parser

Builds:

Program

```

├─ functions[]      fn declarations (compiled first in bytecode)
└─ statements[]    main script body

```

Expression precedence (low → high): equality → comparison → +/− → *// → unary − → primary.

Failures (phase PARSER): unexpected token, bad assignment target, unbalanced ()/{}.

Compiler

Bytecode layout

```

::: infobox{Critical layout} | Offset | Content | | — — | — — — | | 0 | JUMP → skip function bodies | | ...
| Function bodies (LOAD_LOCAL, ...) | | main | Top-level statements | | end | HALT |

```

Without the entry jump, the VM would start inside a function and crash on LOAD_LOCAL. :::

Control flow

Construct	Technique
if / else	JUMP_IF_FALSE, forward JUMP, patch offsets
while	Loop label, condition, body, jump back

Functions

- FunctionMeta: name, entry address, arity
- Parameters → local slots 0...n-1
- Call: push args, CALL index argc
- Return: RETURN restores IP and pushes value

Virtual machine

State

Field	Role
ip_	Current bytecode index
stack_	Operand stack
globals_	File-mode variables (by index)
frames_	Call stack
session_	Optional REPL name → value map

Failsafes

Guard	Result
Stack depth > 65536	Overflow error
> 1M steps	Infinite-loop guard
Divide by zero	VM error (exit 2)
Uninitialized var	Load error
Bad IP	Truncated bytecode error

Opcode reference

Opcode	Stack effect
PUSH_INT / PUSH_BOOL	▢ value
LOAD_VAR / STORE_VAR	global by name index
LOAD_LOCAL / STORE_LOCAL	function slot
ADD ... GE	pop b, a ▢ result
INPUT	▢ stdin value
PRINT	pop ▢ output
JUMP / JUMP_IF_FALSE	control flow
CALL / RETURN	functions
HALT	stop

Trace A — print 1 + 2;

Step	Action	Stack
1	PUSH_INT 1	[1]
2	PUSH_INT 2	[1, 2]
3	ADD	[3]
4	PRINT	[] ▢ prints 3

Trace B — recursive call

```
fn f(n) { if (n <= 1) { return 1; } return n * f(n - 1); }
print f(5);
```

CALL saves return IP, loads locals from stack args, jumps to function address. RETURN pops frame and pushes result for the caller. Final PRINT shows **120**.

Debug workflow

```
./build/cvmpp -d examples/hello.cvm
```

Read output in order: **tokens** □ **AST** □ **bytecode** □ **runtime**.

Module dependencies

```
main → compile → lexer, parser, compiler, vm, ui  
compiler → bytecode, opcode, ast  
vm → bytecode, value
```